Laboratoriya ishi: Kotlin maxsus kalit so‘zlari

Savol 1,

Oddiy funksiya lambda qabul qilingada, har safar uni ishlatish uchun qoshimcha vaqt va xotira sarflanishi mumkin. Masalan, Android ilovasida royxatini aylantirayotganda bu funksiya kop marta ishlansa, ortiqcha xarajatlar yigiladi.

Inline ishlatilsa, Kotlin kodini funksiya chaqirilgan joyga joylashiradi. Shuningu chun lambda bilan bogliq ortiqqcha xarajatlar kamiysihi va dastur tezzroq ishlashi mumkin.

Agar funksiya lambda qabul qilmasa Inline dan unchalik foyda yoq. Chunki kamaytirilgan qoshimcha xarajatlar ham bolmaydi .

Menimcha Inlineni kichik va kop ishlatilgan funksiyalarda qollangan yaxshi. Lekin katta funksiyalarda kopincha qilmagan bolar edim, chunki kod takrorlanib ilovaning hajmi kattalashi mumkin

Savol 2

Foreach Ichida return ishlatilsa u butun funksiyani toxtatishi mumkin. Bu non-local

Return deyiladi. Lekin bu faqat inline funksiyalarda ishlaydi. Oddiy funksiyalarda esa bunday yozish xato.

return@forEach esa faqat hozirgi elementni tashlab keyingisiga otadi break siklni butunlay toxtatadi continue esa hozirgi takrorlanishni otkazib yuboradi. Foreach Ichida break va continue ni oddiy qilip ishlatip bolmaydi.

Masalan. Buyurtma formasida birinchi xató topilsa tekshirishni toxtatish kerak. Telefon raqamlarini tekshirganda esa bosh yoki notogri raqamlarni tashlab ketishi mumkin. Dastlabki 5 ta togri raqamga SMS yuborish uchun oddiy for siklidan foydalanib 5 taga yetganda break bilan toxtatish kerak.

Xulasa qilip aytganda return, break va continue bir-biridan farq qiladi. Ularni vazyatga qarap ishlatish kerak Shunda kod tushinarli va xatosiz boladi.

Savol 3

Inline funksiyada lambda odatda kod ishlayotgan paytda togirdan-togri bajariladi.

Shuning uchun uni keyinroq ishlatish uchun saqlash kerak bolsa noinline dan foydalanish kerak.

Bu vaziyatda ikkita lambda bolsa men yuklash bajaradigan lambdaga noinline qoygan bolardim. Chunki sorov xató bilan tugasa foydalanuvchi “Qayta urinish” tugmasini bosganda shu amalni yana bajarish kerak boladi. Ikkinchi lambda esa faqat yuklash vaqtida ishlatilsa uni inline bolsa ozgartirmaslik kerak.

Agar noinle keraki lambda qoyilmasa uni sqlashda xatolik chiqishi mumkin. Agar ikkala lambdaga ham noinline qoyilsa kod ishlaydi lekin keraksiz joylarda lambda obyektlar yaratilishi mumkin.

Xulosa qilip aytganda noinline ni keyinroq ishlatish uchun saqlash kerak bolgan lambdaga qoyish kerak. Shunday qilganda foydalanuvchi qayta urinsh tugmasini bosganda yuklashni yana takror qilishi mumkin.

Savol 4

Crossline asosan lambda keyinroq boshqa funksiya orqali ishlatilganda kerak boladi. Masalan foydalanuvchi “Tolash” tugmasini ikki marta bosip yuborsa debounce yordamchisi ortiqca bosishini otkazib yubordi va ikkita buyurtma yaratilishining oldini oladi.

Agar lambda Ichida return orqali tashqi funksiyadan chiqishga ruxsat berilsa dastur kutilmagada toxtab qolishi mumkin. Crossinline shu muammoning oldini oladi. U lambdani inline qilishga imkon beradi lekin tashqi funksiyadan return orqali chiqishni taqiqlaydi.

Noinline esa lambdani inline qilinmaydi va uni keyinroq ishlatish uchun saqlashga imkon beradi, crossinline da esa lambda inline bolib mumkin, faqat non-local return ishlatib bolmaydi.

Agar lambda ichidagi amalni erta toxtatish kerak bolsa if sharti yoki return@label dan foydalanish mumkin. Shundan faqat kerakli amal toxtaydi tashqi funksiya esa ishlashda davom etadi.

Savol 5

Kotlin’da generic tur ishlatilganda, dastur ishlayotgan paytda uning aniq turi ko‘pincha saqlanib qolmaydi. Masalan, funksiya T turini qabul qilsa, keyinchalik aynan qaysi klass ekanini bilish qiyin bo‘lishi mumkin. Shuning uchun klassni alohida parametr sifatida berishga to‘g‘ri keladi.

reified shu muammoni hal qiladi. Uning yordamida turni burchakli qavs ichida yozib, klassini alohida uzatmasdan ishlatish mumkin. Masalan, Android’da Activity ochishda yoki JSON ma’lumotlarini kerakli modelga aylantirishda qulaylik yaratadi.

reified faqat inline funksiyalarda ishlaydi, chunki inline funksiya chaqirilgan joyga qo‘yiladi va kompilyator aniq turni bilishi mumkin bo‘ladi. Nothing kabi turdan foydalanib bo‘lmaydi, chunki u oddiy obyekt turi emas, balki hech qachon normal qiymat qaytarmaydigan holatni bildiradi.

Men o‘z loyihalarimda reified dan foydalanardim, chunki u kodni qisqaroq va tushunarli qiladi. Lekin uni faqat haqiqatan ham tur haqida ma’lumot kerak bo‘lgan joylarda ishlatgan bo‘lardim.

Savol 6

Property accessorni inline qilish uchun get() yoki set() oldiga inline yoziladi. Backing field bo‘lsa, inline qilish cheklanishi mumkin, chunki u property qiymatini alohida xotirada saqlaydi. Kompilyator inline accessor kodini chaqirilgan joyga qo‘yadi, bu esa funksiya chaqirish xarajatini kamaytiradi.

Menimcha, tez-tez ishlatiladigan dp ni pikselga aylantirish kabi kichik yordamchilarni inline qilish foydali. Ammo oddiy SharedPreferences amallari uchun bu har doim ham kerak emas, chunki foydasi juda kichik bo‘lishi mumkin.

Savol 7

Public inline funksiya boshqa modulda ishlatilganda uning kodi chaqirilgan joyga ko‘chiriladi. Shu sababli u internal funksiyani to‘g‘ridan-to‘g‘ri chaqira olmaydi, chunki bu modul chegarasini buzishi mumkin. Agar ruxsat berilsa, kutubxona yangilanganda eski kod bilan moslik muammolari paydo bo‘lishi mumkin.

@PublishedApi esa internal funksiyani public inline kod ichida ishlatishga ruxsat beradi. Menimcha, faqat kichik va tez-tez ishlatiladigan funksiyalarni inline qilish kerak, katta yoki murakkab funksiyalarni esa oddiy qoldirgan ma’qul.